

## Pflichtenheft

|                           |                                                                                                              |
|---------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>Projektbezeichnung</b> | Entwicklung einer Terminalanwendung, welche die modulare Installation von Softwarearchitektur automatisiert. |
| <b>Projektleitung</b>     | Nils Gronowski                                                                                               |
| <b>Erstellt am</b>        | 20.03.2026                                                                                                   |
| <b>Letzte Änderung am</b> | 23.03.2026                                                                                                   |
| <b>Status</b>             | [in <b>Bearbeitung</b> /fertiggestellt/pausiert/abgebrochen/Prüfung]                                         |
| <b>Aktuelle Version</b>   | 0.2                                                                                                          |

## Inhalt

### Inhaltsverzeichnis

|                                          |          |
|------------------------------------------|----------|
| <b>1 Einleitung</b>                      | <b>2</b> |
| <b>2 Allgemeines</b>                     | <b>2</b> |
| 2.1 Ziel und Zweck des Dokuments         | 2        |
| 2.2 Ausgangssituation                    | 2        |
| 2.3 Projektbezug                         | 3        |
| 2.4 Abkürzungen                          | 3        |
| 2.5 Teams und Schnittstellen             | 3        |
| <b>3 Konzept</b>                         | <b>3</b> |
| 3.1 Ziele des Anbieters                  | 3        |
| 3.2 Ziele und Nutzen der Anwender:innen  | 3        |
| 3.3 Zielgruppe                           | 4        |
| <b>4 Funktionale Anforderungen</b>       | <b>4</b> |
| 4.1 Start der Anwendung                  | 4        |
| 4.2 Interaktiver Konfigurationsassistent | 4        |
| 4.3 Automatische Projekterstellung       | 4        |
| 4.4 Containerisierung                    | 4        |
| 4.5 Startfähigkeit der Umgebung          | 5        |
| 4.6 Technologieumfang (Prototyp)         | 5        |

|                                             |           |
|---------------------------------------------|-----------|
| <b>5 Nichtfunktionale Anforderungen</b>     | <b>5</b>  |
| 5.1 Allgemeine Anforderungen                | 5         |
| 5.2 Gesetzliche Anforderungen               | 5         |
| 5.3 Technische Anforderungen                | 5         |
| 5.4 Weitere Anforderungen                   | 5         |
| <b>6 Rahmenbedingungen</b>                  | <b>6</b>  |
| 6.1 Zeitplan                                | 6         |
| 6.2 Technische Anforderungen                | 6         |
| 6.3 Problemanalyse                          | 6         |
| 6.4 Qualität                                | 6         |
| <b>7 Liefer- und Abnahmevoraussetzungen</b> | <b>6</b>  |
| 7.1 Projektumfang                           | 6         |
| 7.2 Lieferumfang                            | 7         |
| 7.3 Abnahmekriterien                        | 8         |
| 7.4 Abbruchkriterien                        | 10        |
| <b>8 Anhang</b>                             | <b>11</b> |

## 1 Einleitung

Dieses Pflichtenheft beschreibt die konkrete technische Umsetzung der im Lastenheft definierten Anforderungen zur Entwicklung einer terminalbasierten Anwendung und grenzt den Projektumfang ab.

Ziel ist die Entwicklung eines CLI-Tools, welches automatisiert eine vollständige, containerisierte Entwicklungsumgebung für Webprojekte generiert.

## 2 Allgemeines

### 2.1 Ziel und Zweck des Dokuments

Dieses Pflichtenheft beschreibt die technische Realisierung einer CLI-Anwendung zur automatisierten Projektinitialisierung inklusive Containerisierung.

Es dient als verbindliche Grundlage für die Umsetzung und spätere Abnahme des Projekts.

### 2.2 Ausgangssituation

Der Einstieg in moderne Webentwicklung ist durch viele Technologien und Konfigurationen komplex.

Das System soll:

- manuelle Einrichtung reduzieren
- standardisierte Projektstrukturen schaffen
- schnelle Einsatzbereitschaft gewährleisten

## 2.3 Projektbezug

Das Projekt wird im Rahmen der Ausbildung zum Fachinformatiker für Anwendungsentwicklung durchgeführt und dient als Abschlussprojekt.

## 2.4 Abkürzungen

| Abkürzungen | Bedeutung                         |
|-------------|-----------------------------------|
| CLI         | Command Line Interface            |
| API         | Application Programming Interface |
| DB          | Datenbank                         |
| UI          | User Interface                    |
| Podman      | Container-Plattform               |

## 2.5 Teams und Schnittstellen

Mitwirkende am Projekt werden möglichst genau benannt und Kontaktdaten angelegt.

| Rolle                       | Name           | E-Mail                 |
|-----------------------------|----------------|------------------------|
| Entwickler + Projektmanager | Nils Gronowski | Nils.gronowski@wdr.de  |
| Auftraggeber                | Dirk Kerpen    | Dirk.kerpen@wdr.de     |
| Auftraggeber                | Wolfgang Höfer | Wolfgang.hoefer@wdr.de |

# 3 Konzept

## 3.1 Ziele des Projekts

- Entwicklung eines funktionsfähigen Prototyps
- Demonstration moderner Technologien (Containerisierung, Automatisierung)
- Bereitstellung eines wiederverwendbaren Tools
- Modularität soll das Projekt in Zukunft beliebig erweiterbar machen

## 3.2 Ziele und Nutzen der Anwender:innen

- Schneller Einstieg in Webprojekte
- Reduzierte Einrichtungszeit
- Automatische Konfiguration der Entwicklungsumgebung

### 3.3 Zielgruppe

- Auszubildende
- Einsteiger in die Webentwicklung
- Entwickler mit Fokus auf schnelle Projektstarts oder zum initialisieren von lokalen Tests

## 4 Funktionale Anforderungen

### 4.1 Start der Anwendung

- Umsetzung als CLI-Tool (Im Entwurf: z. B. Node.js / Python)
- Start über Terminalbefehl
- Im Entwurf zu entscheiden: globale Installation via npm oder pip

### 4.2 Interaktiver Konfigurationsassistent

Umsetzung über CLI-Prompts

Benutzer wählt:

- Frontend (z. B. React, Vue)
- Backend (z. B. Node.js, Express)
- Datenbank (z. B. MongoDB, PostgreSQL)
- Validierung der Eingaben

### 4.3 Automatische Projekterstellung

- Generierung einer standardisierten Ordnerstruktur:
  - /frontend
  - /backend
  - /database
  - Podman-compose.yml
- Ausführung von Setup-Befehlen:
  - Z.B. npm init / create-app
  - Installation von Dependencies

### 4.4 Containerisierung

- Automatische Erstellung:
  - Containerfile (Frontend & Backend)
  - Podman-compose.yml
- Definition aller Services:
  - Frontend
  - Backend
  - Datenbank

## 4.5 Startfähigkeit der Umgebung

- Start über:
  - podman-compose up
- Sicherstellung:
  - alle Container starten erfolgreich
  - Services sind erreichbar und Health-Check ist erfolgreich

## 4.6 Technologieumfang (Prototyp)

Mindestens:

- Zwei aktuelle Frontend Frameworks Auswahl
- Zwei aktuelle Backend Frameworks Auswahl
- Eine SQL und eine NoSQL Datenbanken Auswahl

# 5 Nichtfunktionale Anforderungen

## 5.1 Allgemeine Anforderungen

- Intuitive CLI-Bedienung
- Klare Benutzerführung
- Fehlermeldungen verständlich darstellen (ANPASSUNG)

## 5.2 Gesetzliche Anforderungen

- Keine Speicherung personenbezogener Daten

## 5.3 Technische Anforderungen

Lauffähig auf:

- MacOS

Voraussetzung:

- Podman installiert
- Je nach Entwurf: Node.js oder Python installiert

## 5.4 Weitere Anforderungen

Wartbarkeit

- Modularer Codeaufbau
- Trennung von Logik und Konfiguration
- Erweiterbarkeit
- Neue Technologien einfach integrierbar

## 6 Rahmenbedingungen

### 6.1 Zeitplan

| Phase                         | Dauer      |
|-------------------------------|------------|
| Analyse- und Definitionsphase | 8 Stunden  |
| Planungs- und Entwurfsphase   | 10 Stunden |
| Implementierung               | 26 Stunden |
| Testphase                     | 4 Stunden  |
| Abschlussphase                | 4 Stunden  |
| Puffer                        | 4 Stunden  |

### 6.2 Technische Anforderungen

Entwicklungsumgebung:

- VS Code

Technologien:

- Programmiersprache nach Ermessen
- Podman
- Ausschließliche Nutzung von lizenzfreien Softwarepaketen

### 6.3 Problemanalyse

(ANPASSUNG)

| Problem                  | Lösung                    |
|--------------------------|---------------------------|
| Podman nicht installiert | Vorabprüfung + Hinweis    |
| Fehlende Abhängigkeiten  | automatische Installation |
| Portkonflikte            | dynamische Ports          |

### 6.4 Qualität

- Funktionstests der generierten Projekte
- Manuelle Tests der CLI
- Prüfung aller Anforderungen

## 7 Liefer- und Abnahmevoraussetzungen

### 7.1 Projektumfang

Im Projekt enthalten (In-Scope):

Die Entwicklung einer terminalbasierten CLI-Anwendung zur automatisierten Erstellung einer containerisierten Entwicklungsumgebung. Dies umfasst:

- Interaktive Konfiguration von Frontend, Backend und Datenbank (mind. 2 Frontend-, 2 Backend-Optionen, 1 SQL und 1 NoSQL Datenbank)

- Automatische Generierung einer standardisierten Projektstruktur
- Initialisierung der Projekte inklusive Installation von Abhängigkeiten
- Erstellung von Containerfiles und einer podman-compose.yml
- Startfähigkeit der Umgebung über „podman-compose up“
- Sicherstellung der grundlegenden Erreichbarkeit der Services
- Umsetzung als funktionaler Prototyp mit modularer Erweiterbarkeit

Nicht Bestandteil (Out-of-Scope):

- Produktionsreife Systeme oder vollständige Deployment-Pipelines
- Grafische Benutzeroberflächen (nur CLI)
- Cloud- oder Hosting-Integration
- Erweiterte Sicherheitsmechanismen
- Persistente Benutzerverwaltung mit Speicherung von Individueller Konfigurationen

## 7.2 Lieferumfang

Das Projekt gilt als geliefert, wenn folgende Bestandteile vollständig vorliegen:

## 1. CLI-Anwendung

- Lauffähige terminalbasierte Anwendung zur Erstellung containerisierter Entwicklungsumgebungen
- Interaktive Benutzerführung zur Auswahl von:
  - mindestens 2 Frontend-Technologien
  - mindestens 2 Backend-Technologien
  - mindestens 1 SQL- und 1 NoSQL-Datenbank

## 2. Generierte Projektstruktur

- Automatisch erzeugte, standardisierte Verzeichnisstruktur
- Separate Komponenten für Frontend, Backend und Datenbank
- Konsistente und nachvollziehbare Organisation

## 3. Initialisierte Projekte

- Generierte Grundprojekte (Scaffolding) für alle gewählten Technologien
- Automatische Installation aller benötigten Abhängigkeiten
- Startfähige Basisanwendungen

## 4. Containerisierung

- Erstellung von Containerfiles für alle Services
- Generierung einer podman-compose.yml
- Konfiguration der Kommunikation zwischen den Services

## 5. Ausführbare Entwicklungsumgebung

- Start der gesamten Umgebung über „podman-compose up“
- Alle Services laufen innerhalb von Containern

## 6. Dokumentation

- README-Datei mit:
  - Installationsanleitung
  - Beschreibung der Nutzung der CLI
  - Übersicht der unterstützten Technologien
- Kurze technische Beschreibung der Architektur

## 7. Quellcode

- Vollständiger, versionierter Projektcode
- Modularer Aufbau zur Erweiterbarkeit

### 7.3 Abnahmekriterien

Die Abnahme erfolgt ausschließlich, wenn alle folgenden Kriterien erfüllt sind:



### 1. Funktionalität der CLI

- Die CLI startet ohne Fehler
- Benutzer kann interaktiv Frontend, Backend und Datenbank auswählen
- Ungültige Eingaben werden abgefangen und führen nicht zum Absturz

### 2. Projekterstellung

- Nach Abschluss der Konfiguration wird automatisch eine Projektstruktur erzeugt
- Alle erforderlichen Dateien werden generiert
- Die Struktur entspricht der definierten Vorgabe

### 3. Abhängigkeitsinstallation

- Alle benötigten Abhängigkeiten werden automatisch installiert
- Es treten keine Fehler während der Installation auf

### 4. Containerisierung

- Für jeden Service existiert ein gültiges Containerfile
- Die podman-compose.yml ist syntaktisch korrekt
- Alle Services sind korrekt konfiguriert

### 5. Startfähigkeit

- Die Umgebung lässt sich mit „podman-compose up“ starten
- Es treten keine Fehler beim Start der Container auf

### 6. Erreichbarkeit der Services

- Das Frontend ist im Browser erreichbar
- Das Backend liefert gültige Antworten (z. B. API-Endpunkte)
- Die Datenbank ist vom Backend erreichbar

### 7. Stabilität

- Die Anwendung läuft bei mehrfacher Nutzung reproduzierbar

- Es treten keine kritischen Abstürze auf

## 8. Dokumentation

- Eine vollständige README-Datei ist vorhanden
- Ein neuer Nutzer kann die Anwendung anhand der Dokumentation starten und nutzen

## 7.4 Abbruchkriterien

Das Projekt gilt als nicht erfolgreich abgeschlossen bzw. wird abgebrochen, wenn eines oder mehrere der folgenden Kriterien erfüllt sind:

### 1. Grundlegende Funktionsunfähigkeit

- Die CLI-Anwendung lässt sich nicht starten oder ausführen
- Die Anwendung stürzt wiederholt bei der Nutzung ab

### 2. Fehlende Kernfunktionalität

- Keine funktionierende Auswahl von Frontend, Backend und Datenbank möglich
- Die Generierung der Projektstruktur schlägt fehl

### 3. Fehlerhafte Projekterstellung

- Generierte Projekte sind unvollständig oder nicht lauffähig
- Abhängigkeiten können nicht installiert werden

### 4. Fehlgeschlagene Containerisierung

- Containerfiles werden nicht erstellt oder sind fehlerhaft
- Die podman-compose.yml ist ungültig oder nicht ausführbar

### 5. Nicht startbare Umgebung

- „podman-compose up“ führt zu Fehlern und startet die Umgebung nicht
- Container brechen beim Start dauerhaft ab

## 6. Nichterreichbarkeit der Services

- Frontend ist nicht im Browser erreichbar
- Backend liefert keine gültigen Antworten
- Keine Verbindung zwischen Backend und Datenbank möglich

## 7. Fehlende Dokumentation

- Keine oder unzureichende Anleitung zur Nutzung vorhanden
- Projekt kann von Dritten nicht nachvollzogen oder gestartet werden

## 8. Massive Abweichung vom Projektumfang

- Wesentliche In-Scope-Anforderungen wurden nicht umgesetzt
- Erweiterbarkeit ist nicht gegeben (keine modulare Struktur)

## 9. Zeit- oder Ressourcenüberschreitung

- Projekt kann innerhalb der vorgegebenen Zeit nicht fertiggestellt werden
- Kritische Meilensteine werden nicht erreicht

# 8 Anhang

- Lastenheft